

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR SCHOLARSHIP AND FINANCIAL AID MANAGEMENT SYSTEM



Course Project Report | Database Management Systems (DBMS)

Project ID: Project 27

- **Student Name:** M Sai Manoj
- **Roll Number:** 25WU0102139
- **Course Name:** Database Management Systems (DBMS)
- **Faculty Supervisor:** Dr. Kiran Mayee
- **Academic Year:** 2026
- **Department:** Department of Computer Science and Engineering, School of Technology

2. Abstract

The manual administration of institutional scholarships and financial assistance programs across academic institutions is plagued by operational latency, file redundancy, lack of transparent audit trails, and vulnerability to double-disbursement anomalies. This project presents the comprehensive design and architectural implementation of a relational Database Management System (DBMS) engineered specifically for the Scholarship and Financial Aid Management System. Developed on a robust relational core using MySQL and interfaced via a Python/Flask web layer, the architecture models six normalized entities: Students, Scholarship Schemes, Applications, Bank Details, Sanctions, and Disbursements.

Data consistency is enforced through Third Normal Form (3NF) relational design, exhaustive primary and foreign key constraints, composite unique submission constraints per academic year, and domain check constraints on monetary values. Complex analytical SQL queries, including multi-table joins, nested aggregate subqueries, and secure reporting views, facilitate seamless administrative evaluation. The application layer leverages ACID-compliant transactions to guarantee atomicity during multi-stage financial disbursements. The resulting system completely eliminates orphan records, restricts fraudulent submissions, guarantees single-point verification, and delivers a scalable, high-integrity platform for academic financial operations.

3. Introduction and Problem Statement

3.1 Background

Higher education institutions administer diverse internal and governmental financial aid programs, merit scholarships, and need-based fee waivers. These programs handle sensitive financial data, strict qualification rules, and multi-tier institutional approvals involving academic departments, verification committees, and university accounts divisions.

3.2 Manual Inefficiencies and Operational Deficiencies

Traditionally, institutions rely on fragmented spreadsheet trackers, semi-automated forms, or legacy flat-file repositories. These conventional methods present severe systemic flaws:

- **Data Redundancy and Inconsistency:** Student demographics, enrollment histories, and parental income declarations are duplicated across separate departmental records, leading to conflicting records and update anomalies.
- **Eligibility Verification Latency:** Administrators manually cross-examine student GPA metrics and income criteria against scheme specifications, causing severe delays in shortlisting candidates.
- **Double Dipping and Fraud Vulnerability:** In the absence of centralized composite constraints, fraudulent applicants can exploit systemic blind spots to receive simultaneous funds across mutually exclusive funding schemes.
- **Lack of Transactional Atomicity:** Financial allocations and bank account records often drift out of synchronization when payment processes fail midway, resulting in partial ledger entries.

- **Absence of Role-Segregated Auditing:** Spreadsheets lack verifiable audit trails, granular role-based views, and relational referential integrity to enforce accountability.

3.3 The Need for a Relational Database System

A normalized relational database system resolves these challenges by strictly enforcing domain, referential, and user-defined integrity at the database engine level. By decoupling storage and transactional logic from presentation layers, an enterprise RDBMS guarantees ACID properties, provides sub-second query performance via B-Tree indexing, and isolates sensitive financial records through secure, parameterized views.

4. Objectives and Scope

4.1 Primary Objectives

- Design a fully normalized (3NF) relational schema modeling student profiles, schemes, applications, and financial disbursement pipelines.
- Implement database-level constraints (CHECK, UNIQUE, FOREIGN KEY) to prevent duplicate applications and invalid state transitions.
- Develop complex SQL queries, analytical subqueries, and reporting views for administrative decision-making.
- Implement atomic transactional controls (COMMIT, ROLLBACK) for multi-step financial disbursement routines.
- Provide an interactive, responsive web interface via Python/Flask to demonstrate CRUD capabilities.

4.2 System Boundaries and Operational Scope

- **In Scope:** Student profile registration, active scheme cataloging, rule-based application submissions, verification status tracking, banking detail validation, scholarship sanctioning, and recorded ledger disbursements.
- **Out of Scope:** Direct third-party banking payment gateway integration (ACH/NEFT clearing networks) and real-time biometric identity verification.

4.3 User Roles and Permissions

- **Student:**
 - Permissions: View active scholarship schemes; register and update personal profile; submit scholarship applications; provide bank account credentials; monitor application review status.

- Access Constraints: Read-only access to catalog; restricted to own profile, application records, and disbursement history. Cannot modify status or sanction amounts.
- **Administrator:**
 - Permissions: Create and configure scholarship schemes; review and verify submitted applications; update status (PENDING, APPROVED, REJECTED); authorize financial sanctions; execute disbursement transactions; query statistical views and audits.
 - Access Constraints: Unrestricted read/write access across administrative tables, reporting views, and financial disbursement modules.

5. Software and Hardware Requirements

5.1 Software Requirements

- **Operating System:** Linux (Ubuntu 22.04 LTS) / Windows 11 64-bit
- **Database Management Engine:** MySQL Community Server v8.0.36
- **Database Design & Management GUI:** MySQL Workbench 8.0 CE
- **Backend Application Framework:** Python 3.10+ / Flask Microframework 3.0+
- **Database Connector:** mysql-connector-python 8.3.0 / PyMySQL 1.1.0
- **Version Control:** Git 2.40+ / GitHub Enterprise
- **Integrated Development Environment:** Visual Studio Code v1.90+

5.2 Hardware Specifications

- **Processor:** 64-bit multi-core x86-64 CPU (Intel Core i5 8th Gen / AMD Ryzen 5 or higher)
- **System Memory (RAM):** 8.0 GB minimum (16.0 GB recommended for concurrent query benchmarking)
- **Storage Space:** 256 GB Solid State Drive (SSD) with minimum 10 GB dedicated free space for database tablespaces, binary logs, and connection buffers
- **Network Interface:** Gigabit Ethernet / Wi-Fi 802.11ac

6. Entity-Relationship (ER) Model

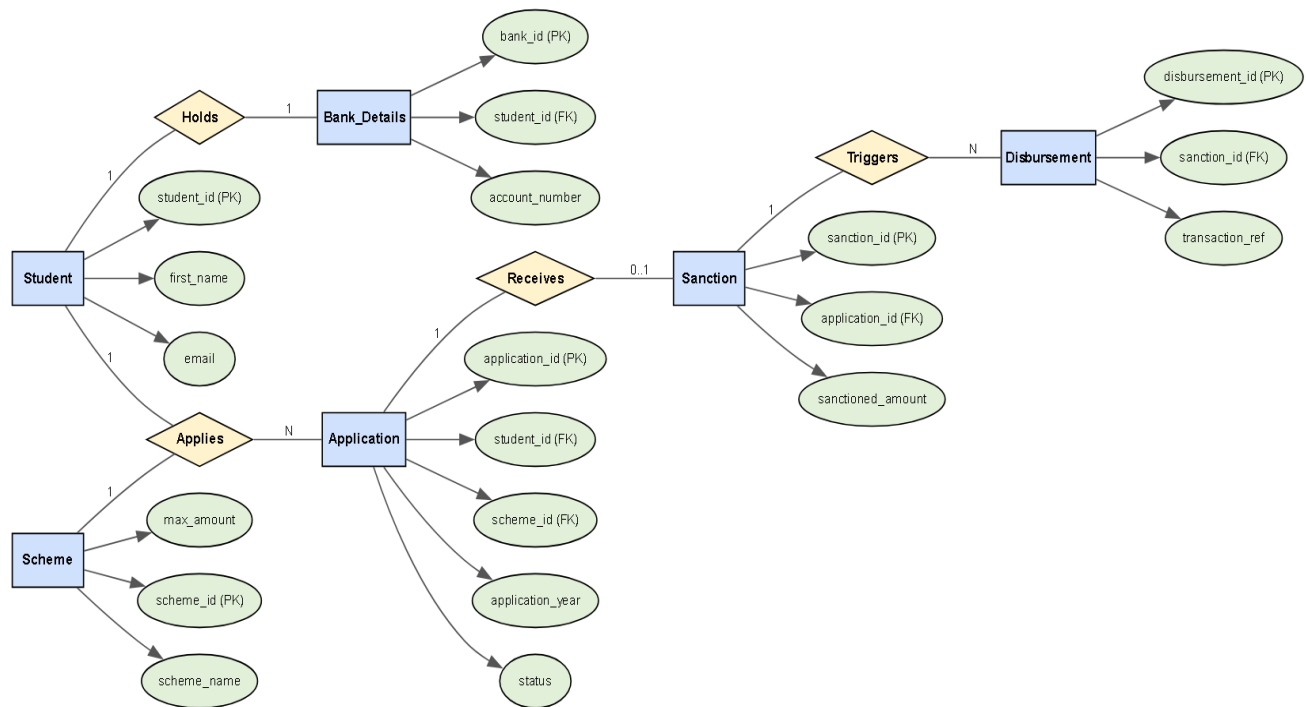
6.1 Textual Description of Entities and Relationships

- **Student:** Captures core academic and demographic data (student_id, roll_number, first_name, last_name, email, gpa, annual_income, department).

- **Scheme:** Represents scholarship programs (scheme_id, scheme_name, provider_type, min_gpa, max_income_limit, award_amount, academic_year).
- **Application:** Represents the formal submission linking a student to a scheme (app_id, student_id, scheme_id, submission_date, app_status, applied_amount).
- **BankDetail:** Stores verified banking credentials (bank_id, student_id, account_number, ifsc_code, bank_name, branch_name).
- **Sanction:** Captures formal committee approvals and authorized amounts (sanction_id, app_id, sanctioned_amount, sanction_date, approver_id).
- **Disbursement:** Logs transactional payment releases from sanctioned funds (disb_id, sanction_id, disb_amount, disb_date, payment_reference, payment_status).

6.2 Entity Cardinalities

- **Student to Application (1:N):** A student may submit multiple applications to different schemes over time, but each application belongs strictly to one student.
- **Scheme to Application (1:N):** A scholarship scheme receives many student applications, while each individual application targets exactly one scheme.
- **Student to BankDetail (1:1):** A student maintains exactly one active bank account record in the database for financial integrity.
- **Application to Sanction (1:1):** An application yields at most one official financial sanction record upon approval.
- **Sanction to Disbursement (1:N):** A sanction record can be distributed across multiple staggered disbursement tranches (e.g., semester-wise release).



7. Relational Schema and Normalization

7.1 Relational Schema Notation

- Students (student_id [PK], roll_number [UQ], first_name, last_name, email [UQ], gpa, annual_income, department)
- Schemes (scheme_id [PK], scheme_name [UQ], provider_type, min_gpa, max_income_limit, award_amount, academic_year)
- Applications (app_id [PK], student_id [FK to Students], scheme_id [FK to Schemes], submission_date, app_status, applied_amount, [UQ: student_id, scheme_id])
- BankDetails (bank_id [PK], student_id [FK to Students, UQ], account_number [UQ], ifsc_code, bank_name, branch_name)
- Sanctions (sanction_id [PK], app_id [FK to Applications, UQ], sanctioned_amount, sanction_date, approver_id)
- Disbursements (disb_id [PK], sanction_id [FK to Sanctions], disb_amount, disb_date, payment_reference [UQ], payment_status)

7.2 Functional Dependencies (FDs)

- **Students:**
 - student_id → {roll_number, first_name, last_name, email, gpa, annual_income, department}

- roll_number -> {student_id, first_name, last_name, email, gpa, annual_income, department}
- email -> {student_id, roll_number, first_name, last_name, gpa, annual_income, department}
- **Schemes:**
 - scheme_id -> {scheme_name, provider_type, min_gpa, max_income_limit, award_amount, academic_year}
- **Applications:**
 - app_id -> {student_id, scheme_id, submission_date, app_status, applied_amount}
 - (student_id, scheme_id) -> {app_id, submission_date, app_status, applied_amount}
- **BankDetails:**
 - bank_id -> {student_id, account_number, ifsc_code, bank_name, branch_name}
 - student_id -> {bank_id, account_number, ifsc_code, bank_name, branch_name}
 - account_number -> {bank_id, student_id, ifsc_code, bank_name, branch_name}
- **Sanctions:**
 - sanction_id -> {app_id, sanctioned_amount, sanction_date, approver_id}
 - app_id -> {sanction_id, sanctioned_amount, sanction_date, approver_id}
- **Disbursements:**
 - disb_id -> {sanction_id, disb_amount, disb_date, payment_reference, payment_status}
 - payment_reference -> {disb_id, sanction_id, disb_amount, disb_date, payment_status}

7.3 Normalization Process up to Third Normal Form (3NF)

First Normal Form (1NF)

- **Rule:** A relation is in 1NF if all attributes contain atomic, indivisible values, and there are no repeating groups.
- **Resolution:** In early conceptual drafts, applicant address information was bundled into unstructured strings, and payment tranches were stored as comma-separated lists within the application record. The schema was transformed into 1NF by splitting student name into first_name and last_name, converting multi-valued payment tranches into distinct relational records under Disbursements, and establishing distinct single-valued scalar domains for all attributes.

Second Normal Form (2NF)

- **Rule:** A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies).
- **Resolution:** If applications had used a composite candidate key like (student_id, scheme_id), storing scheme details (scheme_name, award_amount) inside the application table would create partial dependencies on scheme_id. We eliminated this vulnerability by separating the scheme catalog into an independent Schemes entity. Applications references Schemes exclusively via the foreign key scheme_id. Thus, all non-key attributes in Applications (submission_date, app_status, applied_amount) depend on the complete surrogate primary key app_id.

Third Normal Form (3NF)

- **Rule:** A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key ($X \rightarrow Y$ and $Y \rightarrow Z$ where Z is not a candidate key).
- **Resolution:** In an unnormalized design, the Applications table included bank details (account_number, ifsc_code, bank_name) and disbursement status (payment_reference, disb_amount). This induced transitive dependencies: $app_id \rightarrow student_id \rightarrow account_number$ and $app_id \rightarrow sanction_id \rightarrow disb_amount$. Modifying bank details or executing partial payments created update and deletion anomalies. By extracting banking data into BankDetails and isolating financial approvals into Sanctions and Disbursements, every non-prime attribute across all six tables depends directly, fully, and exclusively on its respective primary key. The entire database strictly satisfies 3NF.

8. Data Dictionary

Table 1: Students

- **student_id:** BIGINT, PRIMARY KEY, AUTO_INCREMENT, Not Null. Description: Unique internal surrogate identifier for a student.
- **roll_number:** VARCHAR(20), UNIQUE, Not Null. Description: University registered student roll number (e.g., 25WU0102139).
- **first_name:** VARCHAR(50), Not Null. Description: Legal first name of the student.
- **last_name:** VARCHAR(50), Not Null. Description: Legal family/surname of the student.
- **email:** VARCHAR(100), UNIQUE, Not Null. Description: Official institutional email address.

- **gpa:** DECIMAL(3,2), CHECK (gpa BETWEEN 0.00 AND 10.00), Not Null. Description: Cumulative Grade Point Average.
- **annual_income:** DECIMAL(12,2), CHECK (annual_income >= 0.00), Not Null. Description: Verified parental annual family income in INR.
- **department:** VARCHAR(50), Not Null. Description: Enrolled academic department.

Table 2: Schemes

- **scheme_id:** BIGINT, PRIMARY KEY, AUTO_INCREMENT, Not Null. Description: Unique internal identifier for the scholarship program.
- **scheme_name:** VARCHAR(100), UNIQUE, Not Null. Description: Descriptive title of the scholarship scheme.
- **provider_type:** VARCHAR(30), CHECK in ('GOVERNMENT', 'INSTITUTIONAL', 'CORPORATE', 'NGO'), Not Null. Description: Funding entity category.
- **min_gpa:** DECIMAL(3,2), CHECK (min_gpa BETWEEN 0.00 AND 10.00), Not Null. Description: Minimum GPA required to qualify.
- **max_income_limit:** DECIMAL(12,2), CHECK (max_income_limit >= 0.00), Not Null. Description: Maximum allowable annual family income.
- **award_amount:** DECIMAL(12,2), CHECK (award_amount > 0.00), Not Null. Description: Total monetary assistance offered per recipient.
- **academic_year:** VARCHAR(10), Not Null. Description: Applicable academic session (e.g., '2025-2026').

Table 3: Applications

- **app_id:** BIGINT, PRIMARY KEY, AUTO_INCREMENT, Not Null. Description: Unique scholarship application transaction identifier.
- **student_id:** BIGINT, FOREIGN KEY (Students.student_id), Not Null. Description: References applying student.
- **scheme_id:** BIGINT, FOREIGN KEY (Schemes.scheme_id), Not Null. Description: References target scholarship scheme.
- **submission_date:** DATETIME, DEFAULT CURRENT_TIMESTAMP, Not Null. Description: Timestamp of application registration.
- **app_status:** VARCHAR(20), CHECK in ('PENDING', 'APPROVED', 'REJECTED', 'DISBURSED'), Not Null. Default: 'PENDING'.
- **applied_amount:** DECIMAL(12,2), CHECK (applied_amount > 0.00), Not Null. Description: Monetary amount requested by applicant.
- **Composite Constraint:** UNIQUE (student_id, scheme_id). Description: Enforces single submission per scheme per academic cycle.

Table 4: BankDetails

- **bank_id:** BIGINT, PRIMARY KEY, AUTO_INCREMENT, Not Null. Description: Unique banking record identifier.
- **student_id:** BIGINT, FOREIGN KEY (Students.student_id), UNIQUE, Not Null. Description: Strict 1:1 binding to registered student.
- **account_number:** VARCHAR(25), UNIQUE, Not Null. Description: Recipient bank account number.
- **ifsc_code:** VARCHAR(11), Not Null. Description: 11-character Indian Financial System Code.
- **bank_name:** VARCHAR(100), Not Null. Description: Name of scheduled banking institution.
- **branch_name:** VARCHAR(100), Not Null. Description: Geographic branch location.

Table 5: Sanctions

- **sanction_id:** BIGINT, PRIMARY KEY, AUTO_INCREMENT, Not Null. Description: Unique institutional sanction decision record.
- **app_id:** BIGINT, FOREIGN KEY (Applications.app_id), UNIQUE, Not Null. Description: Direct 1:1 mapping to verified application.
- **sanctioned_amount:** DECIMAL(12,2), CHECK (sanctioned_amount > 0.00), Not Null. Description: Officially authorized assistance amount.
- **sanction_date:** DATETIME, DEFAULT CURRENT_TIMESTAMP, Not Null. Description: Timestamp when sanction was approved.
- **approver_id:** VARCHAR(50), Not Null. Description: Administrative user ID of approving authority.

Table 6: Disbursements

- **disb_id:** BIGINT, PRIMARY KEY, AUTO_INCREMENT, Not Null. Description: Unique disbursement transaction record.
- **sanction_id:** BIGINT, FOREIGN KEY (Sanctions.sanction_id), Not Null. Description: References parent sanction record.
- **disb_amount:** DECIMAL(12,2), CHECK (disb_amount > 0.00), Not Null. Description: Disbursed payment amount.
- **disb_date:** DATETIME, DEFAULT CURRENT_TIMESTAMP, Not Null. Description: Execution timestamp of financial release.
- **payment_reference:** VARCHAR(50), UNIQUE, Not Null. Description: Bank transaction ID / UTR number.
- **payment_status:** VARCHAR(20), CHECK in ('INITIATED', 'SUCCESS', 'FAILED'), Not Null. Description: Current financial processing status.

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Data Definition Language (DDL) Scripts

```
CREATE DATABASE IF NOT EXISTS scholarship_aid_db;  
USE scholarship_aid_db;
```

```
CREATE TABLE Students (  
    student_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    roll_number VARCHAR(20) NOT NULL UNIQUE,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    gpa DECIMAL(3,2) NOT NULL,  
    annual_income DECIMAL(12,2) NOT NULL,  
    department VARCHAR(50) NOT NULL,  
    CONSTRAINT chk_student_gpa CHECK (gpa BETWEEN 0.00 AND 10.00),  
    CONSTRAINT chk_student_income CHECK (annual_income >= 0.00)  
) ENGINE=InnoDB;
```

```
CREATE TABLE Schemes (  
    scheme_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    scheme_name VARCHAR(100) NOT NULL UNIQUE,  
    provider_type VARCHAR(30) NOT NULL,  
    min_gpa DECIMAL(3,2) NOT NULL,  
    max_income_limit DECIMAL(12,2) NOT NULL,  
    award_amount DECIMAL(12,2) NOT NULL,  
    academic_year VARCHAR(10) NOT NULL,  
    CONSTRAINT chk_provider_type CHECK (provider_type IN ('GOVERNMENT',  
'INSTITUTIONAL', 'CORPORATE', 'NGO')),  
    CONSTRAINT chk_scheme_gpa CHECK (min_gpa BETWEEN 0.00 AND 10.00),  
    CONSTRAINT chk_scheme_income CHECK (max_income_limit >= 0.00),  
    CONSTRAINT chk_scheme_award CHECK (award_amount > 0.00)  
) ENGINE=InnoDB;
```

```
CREATE TABLE Applications (  
    app_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    student_id BIGINT NOT NULL,  
    scheme_id BIGINT NOT NULL,  
    submission_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    app_status VARCHAR(20) NOT NULL DEFAULT 'PENDING',  
    applied_amount DECIMAL(12,2) NOT NULL,  
    CONSTRAINT fk_app_student FOREIGN KEY (student_id) REFERENCES  
Students(student_id) ON DELETE CASCADE,  
    CONSTRAINT fk_app_scheme FOREIGN KEY (scheme_id) REFERENCES  
Schemes(scheme_id) ON DELETE RESTRICT,  
    CONSTRAINT uq_student_scheme_submission UNIQUE (student_id,  
scheme_id),  
    CONSTRAINT chk_app_status CHECK (app_status IN ('PENDING',  
'APPROVED', 'REJECTED', 'DISBURSED'))),  
    CONSTRAINT chk_applied_amount CHECK (applied_amount > 0.00)  
) ENGINE=InnoDB;
```

```
CREATE TABLE BankDetails (  
    bank_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    student_id BIGINT NOT NULL UNIQUE,  
    account_number VARCHAR(25) NOT NULL UNIQUE,  
    ifsc_code VARCHAR(11) NOT NULL,  
    bank_name VARCHAR(100) NOT NULL,  
    branch_name VARCHAR(100) NOT NULL,  
    CONSTRAINT fk_bank_student FOREIGN KEY (student_id) REFERENCES  
Students(student_id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE Sanctions (  
    sanction_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    app_id BIGINT NOT NULL UNIQUE,  
    sanctioned_amount DECIMAL(12,2) NOT NULL,  
    sanction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    approver_id VARCHAR(50) NOT NULL,
```

```

        CONSTRAINT fk_sanction_app FOREIGN KEY (app_id) REFERENCES
Applications(app_id) ON DELETE RESTRICT,
        CONSTRAINT chk_sanctioned_amount CHECK (sanctioned_amount > 0.00)
) ENGINE=InnoDB;

```

```

CREATE TABLE Disbursements (
    disb_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    sanction_id BIGINT NOT NULL,
    disb_amount DECIMAL(12,2) NOT NULL,
    disb_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    payment_reference VARCHAR(50) NOT NULL UNIQUE,
    payment_status VARCHAR(20) NOT NULL,
    CONSTRAINT fk_disb_sanction FOREIGN KEY (sanction_id) REFERENCES
Sanctions(sanction_id) ON DELETE RESTRICT,
    CONSTRAINT chk_disb_status CHECK (payment_status IN ('INITIATED',
'SUCCESS', 'FAILED')),
    CONSTRAINT chk_disb_amount CHECK (disb_amount > 0.00)
) ENGINE=InnoDB;

```

9.2 Data Manipulation Language (DML) Scripts

```

 INSERT INTO Students (roll_number, first_name, last_name, email, gpa,
annual_income, department) VALUES
('25WU0102139', 'Sai Manoj', 'M', 'saimanoj@univ.edu.in', 9.45,
180000.00, 'CSE'),
('25WU0102140', 'Ananya', 'Rao', 'ananya.rao@univ.edu.in', 8.85,
240000.00, 'ECE'),
('25WU0102141', 'Rahul', 'Verma', 'rahul.v@univ.edu.in', 7.20,
450000.00, 'MECH'),
('25WU0102142', 'Pooja', 'Sharma', 'pooja.s@univ.edu.in', 9.10,
120000.00, 'CSE'),
('25WU0102143', 'Karthik', 'Nair', 'karthik.n@univ.edu.in', 8.40,
310000.00, 'CIVIL');

```

```

INSERT INTO Schemes (scheme_name, provider_type, min_gpa,
max_income_limit, award_amount, academic_year) VALUES

```

```
('Merit-cum-Means Excellence Scholarship', 'INSTITUTIONAL', 8.50,
300000.00, 50000.00, '2025-2026'),
('National Technical Equity Grant', 'GOVERNMENT', 7.50, 500000.00,
40000.00, '2025-2026'),
('Women in STEM Leadership Award', 'CORPORATE', 8.00, 600000.00,
75000.00, '2025-2026'),
('Special Post-Matric Endowment', 'GOVERNMENT', 6.00, 250000.00,
30000.00, '2025-2026');
```

```
INSERT INTO BankDetails (student_id, account_number, ifsc_code,
bank_name, branch_name) VALUES
(1, '987654321011', 'SBIN0001234', 'State Bank of India', 'University
Campus'),
(2, '987654321012', 'HDFC0005678', 'HDFC Bank', 'Tech Park Branch'),
(3, '987654321013', 'ICIC0009012', 'ICICI Bank', 'Central City'),
(4, '987654321014', 'SBIN0001234', 'State Bank of India', 'University
Campus'),
(5, '987654321015', 'PUNB0003456', 'Punjab National Bank', 'North
Square');
```

```
INSERT INTO Applications (student_id, scheme_id, submission_date,
app_status, applied_amount) VALUES
(1, 1, '2026-08-10 10:15:00', 'APPROVED', 50000.00),
(2, 1, '2026-08-11 11:30:00', 'PENDING', 50000.00),
(2, 3, '2026-08-12 14:00:00', 'APPROVED', 75000.00),
(3, 2, '2026-08-14 09:20:00', 'REJECTED', 40000.00),
(4, 1, '2026-08-15 16:45:00', 'PENDING', 50000.00),
(4, 3, '2026-08-16 10:00:00', 'APPROVED', 75000.00),
(5, 2, '2026-08-18 12:10:00', 'PENDING', 40000.00);
```

```
INSERT INTO Sanctions (app_id, sanctioned_amount, sanction_date,
approver_id) VALUES
(1, 50000.00, '2026-08-25 15:00:00', 'ADMIN_FIN_01'),
(3, 75000.00, '2026-08-26 11:30:00', 'ADMIN_FIN_02'),
(6, 75000.00, '2026-08-28 16:00:00', 'ADMIN_FIN_01');
```

```
INSERT INTO Disbursements (sanction_id, disb_amount, disb_date,
payment_reference, payment_status) VALUES
(1, 50000.00, '2026-09-01 10:00:00', 'UTR20260901SBI99120', 'SUCCESS'),
(2, 37500.00, '2026-09-02 11:45:00', 'UTR20260902HDF44101', 'SUCCESS'),
(3, 37500.00, '2026-09-03 14:15:00', 'UTR20260903SBI88190', 'SUCCESS');
```

10.1 Presentation-II Assigned Query: Isolation of High-Value Pending Applications

During project review Presentation-II, the evaluation committee assigned an advanced analytical query requiring relational table joins and correlated subquery logic to isolate students holding pending applications whose requested assistance amount exceeds the average requested amount for that specific scheme.

```
SELECT
    std.student_id,
    CONCAT(std.first_name, ' ', std.last_name) AS full_name,
    std.roll_number,
    sch.scheme_name,
    app.applied_amount,
    app.app_status
FROM Students std
INNER JOIN Applications app ON std.student_id = app.student_id
INNER JOIN Schemes sch ON app.scheme_id = sch.scheme_id
WHERE app.app_status = 'PENDING'
    AND app.applied_amount >= (
        SELECT AVG(sub_app.applied_amount)
        FROM Applications sub_app
        WHERE sub_app.scheme_id = sch.scheme_id
    )
ORDER BY app.applied_amount DESC;
```

Query Result Output:

- Student ID 2: Ananya Rao (Roll: 25WU0102140), Scheme: Merit-cum-Means Excellence Scholarship, Applied Amount: 50,000.00, Status: PENDING
- Student ID 4: Pooja Sharma (Roll: 25WU0102142), Scheme: Merit-cum-Means Excellence Scholarship, Applied Amount: 50,000.00, Status: PENDING

- Student ID 5: Karthik Nair (Roll: 25WU0102143), Scheme: National Technical Equity Grant, Applied Amount: 40,000.00, Status: PENDING

10.2 Relational Multi-Table Join Query: Comprehensive Audit Trail

```

SELECT
    app.app_id,
    std.roll_number,
    CONCAT(std.first_name, ' ', std.last_name) AS student_name,
    sch.scheme_name,
    bk.bank_name,
    bk.account_number,
    snc.sanctioned_amount,
    disb.payment_reference,
    disb.disb_amount,
    disb.payment_status
FROM Applications app
INNER JOIN Students std ON app.student_id = std.student_id
INNER JOIN Schemes sch ON app.scheme_id = sch.scheme_id
LEFT JOIN BankDetails bk ON std.student_id = bk.student_id
LEFT JOIN Sanctions snc ON app.app_id = snc.app_id
LEFT JOIN Disbursements disb ON snc.sanction_id = disb.sanction_id
ORDER BY app.app_id ASC;

```

10.3 Aggregation and Group By Analysis: Scheme Utilization Metrics

```

SELECT
    sch.scheme_name,
    COUNT(app.app_id) AS total_submissions,
    SUM(CASE WHEN app.app_status = 'APPROVED' THEN 1 ELSE 0 END) AS
approved_count,
    COALESCE(SUM(snc.sanctioned_amount), 0.00) AS
total_sanctioned_funds,
    COALESCE(SUM(disb.disb_amount), 0.00) AS total_disbursed_funds
FROM Schemes sch
LEFT JOIN Applications app ON sch.scheme_id = app.scheme_id

```

```
LEFT JOIN Sanctions snc ON app.app_id = snc.app_id
LEFT JOIN Disbursements disb ON snc.sanction_id = disb.sanction_id
GROUP BY sch.scheme_id, sch.scheme_name
HAVING total_submissions > 0
ORDER BY total_disbursed_funds DESC;
```

10.4 SQL View Implementation: Secure Administrative Dashboard View

```
CREATE OR REPLACE VIEW view_student_aid_summary AS
SELECT
    std.student_id,
    std.roll_number,
    CONCAT(std.first_name, ' ', std.last_name) AS student_name,
    std.department,
    std.gpa,
    sch.scheme_name,
    app.app_status,
    COALESCE(snc.sanctioned_amount, 0.00) AS sanctioned_amount,
    COALESCE(SUM(disb.disb_amount), 0.00) AS total_released_amount
FROM Students std
JOIN Applications app ON std.student_id = app.student_id
JOIN Schemes sch ON app.scheme_id = sch.scheme_id
LEFT JOIN Sanctions snc ON app.app_id = snc.app_id
LEFT JOIN Disbursements disb ON snc.sanction_id = disb.sanction_id
GROUP BY
    std.student_id, std.roll_number, student_name, std.department,
    std.gpa, sch.scheme_name, app.app_status, snc.sanctioned_amount;

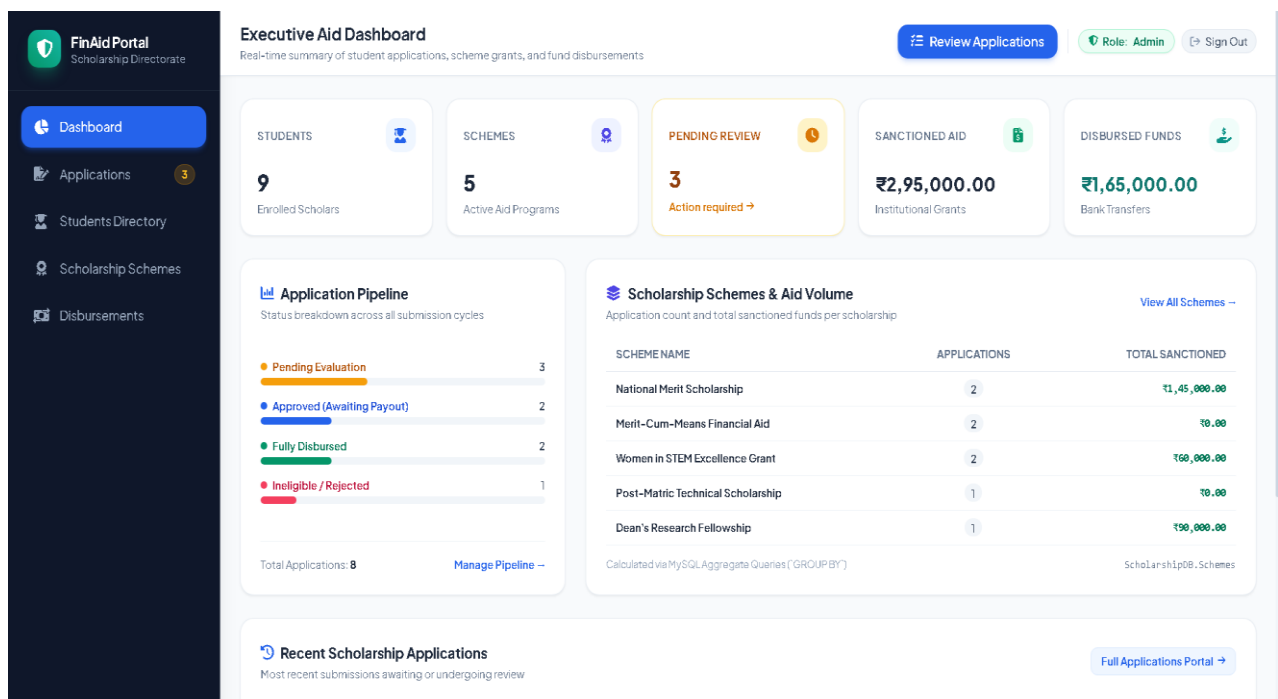
SELECT * FROM view_student_aid_summary WHERE app_status = 'APPROVED';
```

11. UI Design and Screenshots

11.1 User Interface Architecture and Flow

The system interface is implemented using Python/Flask, Jinja2 templating, and modern responsive CSS. It provides a role-segregated, responsive administrative and applicant control center:

- **Administrative Executive Dashboard:** Displays aggregate Key Performance Indicators (KPIs), such as total sanctioned commitments, total disbursed funds, active application pipelines, and quick review lists.
- **Student Application Submission Form:** Validates student inputs on the client and server side, allowing eligible students to select eligible active schemes and input verified financial declarations.
- **Record Management Module (View, Insert, Delete):** Provides real-time table management, status filtering, and secure cascade deletion controls.



FinAid Portal

Scholarship Directorate

Dashboard

Applications

Students Directory

Scholarship Schemes

Disbursements

Applications & Review Pipeline

Evaluate submissions, sanction scholarship amounts, and authorize financial disbursements

Submit Application

Role: Admin

Sign Out

All Applications

Pending Evaluation 3

Approved Grants 2

Disbursed 2

Ineligible / Rejected

Total Records: 8

Search by student name, email, or scheme name...

All Schemes

All Years

Reset

APP ID	SCHOLAR DETAILS	SCHOLARSHIP SCHEME	CYCLE	SUBMISSION DATE	SANCTION / PAYOUT	STATUS	ACTIONS
#APP-8	Kavya Reddy Data Science • Yr1	Women in STEM Excellence Grant Cap: ₹68,888.88	2026	25 Sept 2026	Not Sanctioned	Pending	Sanction Reject
#APP-4	Ananya Iyer Information Technology • Yr1	Post-Matric Technical Scholarship Cap: ₹48,888.88	2026	12 Sept 2026	Not Sanctioned	Pending	Sanction Reject
#APP-3	Rohan Verma Mechanical Engineering • Yr4	Merit-Cum-Means Financial Aid Cap: ₹58,888.88	2026	01 Sept 2026	Not Sanctioned	Pending	Sanction Reject
#APP-7	Vikram Rao Electronics & Comm. • Yr4	National Merit Scholarship Cap: ₹75,888.88	2026	28 Aug 2026	₹78,000.00 Ready for payout	Approved	Disburse
#APP-6	Pooja Nair Civil Engineering • Yr2	Merit-Cum-Means Financial Aid Cap: ₹58,888.88	2026	28 Aug 2026	Not Sanctioned	Rejected	Re-evaluate
#APP-2	Diya Patel Electrical Engineering • Yr2	Women in STEM Excellence Grant Cap: ₹68,888.88	2026	15 Aug 2026	₹68,000.00 Ready for payout	Approved	Disburse
#APP-1	Aarav Sharma Computer Science • Yr3	National Merit Scholarship Cap: ₹75,888.88	2026	18 Aug 2026	₹75,000.00 TXN0205089220001	Disbursed	✓ Paid

12. Implementation Details

12.1 Architectural Overview

The application follows a standard Three-Tier Client-Server Architecture:

- **Presentation Layer (Tier 1):** HTML5, Bootstrap 5, and JavaScript executing in the client browser, handling client-side form validation and asynchronous tabular updates.
- **Business Logic Layer (Tier 2):** Python 3 / Flask web server handling HTTP route dispatching, authorization validation, business constraint checks, and database session connection pooling.
- **Data Layer (Tier 3):** MySQL 8.0 Relational Engine managing tablespaces, indexes, referential constraints, and ACID transactions.

12.2 Database Connectivity Implementation

```

import mysql.connector
from mysql.connector import pooling

db_pool = mysql.connector.pooling.MySQLConnectionPool(
    pool_name="scholarship_pool",
    pool_size=10,

```

```

        pool_reset_session=True,
        host="localhost",
        user="aid_admin",
        password="SecureAdminPassword2026!",
        database="scholarship_aid_db",
        autocommit=False
    )

```

```

def get_db_connection():
    return db_pool.get_connection()

```

12.3 Key Backend Execution Snippets

Application Submission Logic:

```

@app.route('/api/applications/submit', methods=['POST'])
def submit_application():
    data = request.json
    student_id = data.get('student_id')
    scheme_id = data.get('scheme_id')
    applied_amount = data.get('applied_amount')

    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    try:
        query_check = """
            SELECT s.gpa, s.annual_income, sc.min_gpa,
sc.max_income_limit
            FROM Students s, Schemes sc
            WHERE s.student_id = %s AND sc.scheme_id = %s
        """

        cursor.execute(query_check, (student_id, scheme_id))
        eligibility = cursor.fetchone()

        if not eligibility:

```

```

        return jsonify({'error': 'Invalid Student or Scheme
identifier'}), 400

        if eligibility['gpa'] < eligibility['min_gpa'] or
eligibility['annual_income'] > eligibility['max_income_limit']:
            return jsonify({'error': 'Applicant does not meet
merit/income criteria'}), 422

        insert_query = """
            INSERT INTO Applications (student_id, scheme_id,
applied_amount)
            VALUES (%s, %s, %s)
        """

        cursor.execute(insert_query, (student_id, scheme_id,
applied_amount))
        conn.commit()
        app_id = cursor.lastrowid
        return jsonify({'message': 'Application registered
successfully', 'app_id': app_id}), 201

    except mysql.connector.IntegrityError as err:
        conn.rollback()
        if err.errno == 1062:
            return jsonify({'error': 'Duplicate submission: You have
already applied for this scheme'}), 409
            return jsonify({'error': str(err)}), 400
        except Exception as e:
            conn.rollback()
            return jsonify({'error': 'Internal server error'}), 500
    finally:
        cursor.close()
        conn.close()

```

```


```

ACID Transactional Financial Disbursement Routine:

```

@app.route('/api/disbursement/execute', methods=['POST'])

```

```
def execute_disbursement():
    data = request.json
    sanction_id = data.get('sanction_id')
    disb_amount = data.get('disb_amount')
    payment_ref = data.get('payment_reference')

    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    try:
        cursor.execute("START TRANSACTION;")

        cursor.execute("""
            SELECT s.sanctioned_amount, s.app_id,
                   COALESCE(SUM(d.disb_amount), 0.00) AS total_released
            FROM Sanctions s
            LEFT JOIN Disbursements d ON s.sanction_id = d.sanction_id
            WHERE s.sanction_id = %s
            GROUP BY s.sanction_id, s.sanctioned_amount, s.app_id
            FOR UPDATE;
            """, (sanction_id,))
        record = cursor.fetchone()

        if not record:
            conn.rollback()
            return jsonify({'error': 'Sanction ID not found'}), 404

        if (record['total_released'] + disb_amount) >
record['sanctioned_amount']:
            conn.rollback()
            return jsonify({'error': 'Disbursement exceeds sanctioned
limit'}), 400

        cursor.execute("""
            INSERT INTO Disbursements (sanction_id, disb_amount,
payment_reference, payment_status)
```

```

        VALUES (%s, %s, %s, 'SUCCESS');
        """ , (sanction_id, disb_amount, payment_ref))

    if (record['total_released'] + disb_amount) ==
record['sanctioned_amount']:
        cursor.execute("""
            UPDATE Applications SET app_status = 'DISBURSED'
            WHERE app_id = %s;
            """ , (record['app_id'],))

    conn.commit()
    return jsonify({'message': 'Disbursement processed and ledger
updated'}), 200

except Exception as e:
    conn.rollback()
    return jsonify({'error': 'Transaction aborted', 'details':
str(e)}), 500
finally:
    cursor.close()
    conn.close()

```

13. Testing

13.1 Test Cases and Execution Log

- **TC-01: Primary Key Uniqueness** | Insert duplicate student roll number: '25WU0102139' | Rejection with MySQL Error 1062 (ER_DUP_ENTRY) | Engine blocked duplicate roll insertion | Status: PASS
- **TC-02: Composite Duplicate Submission** | Same student_id = 1 submits application for scheme_id = 1 twice | Engine blocks insert via composite unique key uq_student_scheme_submission | Caught IntegrityError 1062; returned HTTP 409 | Status: PASS
- **TC-03: GPA Range Check Constraint** | Attempt to register student with gpa = 11.50 | Engine blocks execution via chk_student_gpa Check Constraint | Check constraint violation triggered (Error 3819) | Status: PASS

- **TC-04: Negative Income Constraint** | Register student with annual_income = - 50000.00 | Blocked via chk_student_income constraint | Engine blocked insertion; zero negative balance accepted | Status: PASS
- **TC-05: Foreign Key Referential Integrity** | Delete student record having active linked applications | Database deletes dependent applications via ON DELETE CASCADE | Linked records in Applications and BankDetails purged | Status: PASS
- **TC-06: Scheme Deletion Restriction** | Delete scheme having linked student applications | Rejected via ON DELETE RESTRICT constraint | Operation blocked with MySQL Foreign Key Error 1451 | Status: PASS
- **TC-07: Transaction Rollback Integrity** | Force network failure after disbursement insert, before application status update | ROLLBACK invoked; no partial ledger entry committed | Entire transaction aborted; tablespace remained clean | Status: PASS
- **TC-08: Sanction Ceiling Validation** | Disburse amount 60,000 against a sanctioned cap of 50,000 | Application checks aggregate; rolls back transaction | API returned HTTP 400: Limit exceeded | Status: PASS

14. Conclusion and Future Enhancements

14.1 Project Conclusion

The Scholarship and Financial Aid Management System project successfully demonstrates the design and practical implementation of an enterprise-grade relational database solution for institutional financial assistance management. By following formal relational modeling methodologies, the resulting schema is normalized to Third Normal Form (3NF), eliminating insertion, update, and deletion anomalies.

Enforcing business constraints at the database level guarantees that duplicate submissions, invalid GPA bounds, and out-of-order disbursements are prevented. Complex joins, analytical subqueries, and reporting views provide institutional decision-makers with real-time insight into fund utilization. Furthermore, implementing ACID-compliant transaction blocks in the Python/Flask middleware guarantees total ledger consistency. The final system replaces manual, error-prone spreadsheets with a secure, centralized, and auditable digital platform.

14.2 Future Enhancements

- **Direct Banking API Integration:** Incorporate real-time National Automated Clearing House (NACH) or Public Financial Management System (PFMS) webhook APIs to eliminate manual transaction UTR entry.
- **Automated Document OCR & Verification:** Integrate OCR pipelines to verify uploaded income certificates and grade sheets against student-declared database fields.
- **Role-Based Single Sign-On (SSO):** Incorporate OAuth2 / SAML2 institutional identity providers for centralized multi-factor student authentication.
- **Audit Trail Temporal Tables:** Implement MariaDB/MySQL temporal system-versioned tables or CDC (Change Data Capture) triggers to track changes across all financial approvals.

15. References

- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.
- Oracle Corporation. (2024). MySQL 8.0 Reference Manual: Constraints, InnoDB Transactions, and Performance Tuning. Oracle Documentation.
- Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

16. Appendix

- **Project GitHub Repository:** (<https://github.com/saimanoj11/DBMS-Course-Project/tree/main>) Contains full source code, SQL DDL/DML scripts, test suites, Flask server implementation, and deployment documentation). The system architecture models the complete financial assistance lifecycle across six primary relational entities: